

LAC-NAS: Lyapunov-Guided Adaptive Chaotic Neural Architecture Search

Ian M. Breda

¹Faculdade de Ciências – Universidade Estadual Paulista (UNESP)
Bauru, SP – Brasil

Abstract. *This paper investigates the application of chaos theory-based genetic algorithms to Neural Architecture Search (NAS) in the context of optimizing convolutional neural network topologies. Using a public benchmark dataset, a comparison is performed between the proposed approach, LAC-NAS (Lyapunov-controlled Chaotic NAS), and a traditional genetic algorithm, evaluating their effectiveness in discovering high-performance architectures. The study explores how adaptive search strategies, controlled by principles of dynamical systems, can enhance the optimization process in NAS through a dynamic balance between exploration and exploitation, contributing to the mitigation of premature convergence and the discovery of more robust solutions.*

Resumo. *Este artigo investiga a aplicação de algoritmos genéticos baseados em teoria do caos à Busca Automatizada de Arquiteturas Neurais (NAS), no contexto da otimização de topologias para redes neurais convolucionais. Utilizando um dataset público de benchmark, realiza-se uma comparação entre a abordagem proposta, o LAC-NAS (Lyapunov-controlled Chaotic NAS), e um algoritmo genético tradicional, avaliando a eficácia de ambos na descoberta de arquiteturas de alto desempenho. O estudo explora como estratégias de busca adaptativas, controladas por princípios de sistemas dinâmicos, podem aprimorar o processo de otimização em NAS por meio de um balanço dinâmico entre exploração e exploração, contribuindo para a mitigação da convergência prematura e para a descoberta de soluções mais robustas.*

1. Introdução

Na última década, o aprendizado profundo se consolidou como o paradigma dominante em aprendizado de máquina, impulsionando avanços em visão computacional, processamento de linguagem natural, reconhecimento de fala e análise de dados tabulares. Embora o aumento no volume de dados e na capacidade computacional tenham sido essenciais, o projeto de arquiteturas de redes neurais de alto desempenho foi um fator crucial para esse sucesso [White et al. 2023]. Contudo, o desenho manual dessas arquiteturas é um processo demorado, caro e que exige profundo conhecimento técnico [Sun et al. 2020].

Para superar esse gargalo, surgiu o campo da Busca Automatizada de Arquiteturas Neurais (Neural Architecture Search - NAS), que visa automatizar o processo de desenho de redes neurais para uma determinada tarefa. O NAS já demonstrou ser capaz de superar arquiteturas projetadas por especialistas humanos em diversas aplicações complexas [Liu et al. 2019], o que gerou um crescimento exponencial no interesse e na pesquisa na área, com mais de mil artigos publicados apenas nos últimos anos.

1.1. Definição do problema

O problema central que o NAS aborda é a otimização da topologia de uma rede neural, uma tarefa significativamente mais complexa do que a otimização de hiperparâmetros (HPO) tradicional. Enquanto a HPO geralmente lida com um conjunto de parâmetros contínuos ou categóricos independentes (como taxa de aprendizado ou tamanho do lote), o NAS opera em um espaço de busca discreto, vasto e estruturado, onde as escolhas são condicionais e interligadas [White et al. 2023].

Formalmente, o objetivo do NAS é encontrar, dentro de um espaço de busca $\mathcal{A}$ e um orçamento computacional, uma arquitetura a que minimize a perda de validação, conforme a expressão:

$$\min_{a \in \mathcal{A}} \mathcal{L}_{\text{val}}(w^*(a), a) \text{ s.t. } w^*(a) = \arg \min_w \mathcal{L}_{\text{train}}(w, a). \quad (1)$$

Para lidar com essa complexidade, o problema é tipicamente decomposto em três componentes principais:

- Espaço de Busca: Define o conjunto de todas as arquiteturas possíveis que podem ser projetadas, incluindo os tipos de operações (convoluções, pooling etc.), suas conexões e a topologia geral.
- Estratégia de Busca: O algoritmo de otimização utilizado para explorar o espaço de busca e encontrar a melhor arquitetura. Estratégias comuns incluem aprendizado por reforço, otimização Bayesiana e algoritmos evolucionários.
- Estratégia de Avaliação: O método usado para estimar a qualidade de uma arquitetura candidata, idealmente sem a necessidade de treiná-la completamente até a convergência, o que acelera drasticamente a busca.

1.2. Solução proposta

Dentre as diversas estratégias de busca, os Algoritmos Genéticos se destacam por sua eficácia. No entanto, sua versão tradicional frequentemente sofre com a convergência prematura, causada por um desequilíbrio entre a exploração (busca por novas soluções) e a exploração (refinamento de boas soluções existentes), geralmente ditado por taxas de crossover e mutação fixas.

Este artigo propõe uma solução para essa limitação: o LAC-NAS (Lyapunov-controlled Chaotic Neural Architecture Search). Esta abordagem emprega um algoritmo genético cujas taxas de crossover e mutação são geradas dinamicamente pelo mapa logístico, um sistema caótico. O nível de caos desse sistema é, por sua vez, controlado pelo expoente de Lyapunov, que atua como um sensor da estagnação da população. Isso permite que o algoritmo se adapte, intensificando a exploração quando necessário e focando na exploração ao encontrar regiões promissoras, superando as limitações do AG tradicional. O objetivo deste trabalho é realizar uma análise comparativa entre o LAC-NAS e uma abordagem genética convencional para demonstrar a eficácia e a robustez da nova proposta.

2. Metodologia

Nesta seção, são descritos os componentes e procedimentos experimentais utilizados para avaliar a abordagem LAC-NAS em comparação com um algoritmo genético tradicional. Será detalhada a base de dados, o espaço de busca de arquiteturas, as estratégias de otimização e as ferramentas empregadas.

2.1. Base de dados

O conjunto de dados utilizado para o treinamento, validação e teste das arquiteturas neurais foi o CIFAR-10. Este é um benchmark amplamente reconhecido e utilizado na comunidade de visão computacional, o que facilita a comparação e a reprodutibilidade dos resultados.

As imagens do dataset foram submetidas a um pipeline de pré-processamento padrão para modelos baseados em PyTorch. As transformações aplicadas a cada imagem foram:

- **Conversão para Tensor:** As imagens foram convertidas de seu formato original para tensores do PyTorch.
- **Normalização:** Os valores dos pixels de cada canal de cor (RGB) foram normalizados para o intervalo $[-1, 1]$. Isso foi feito subtraindo uma média de 0.5 e dividindo por um desvio padrão de 0.5.

Para o processo de treinamento e avaliação, o conjunto de dados original foi particionado da seguinte maneira:

- O conjunto de treinamento original foi dividido em dois novos subconjuntos: 90% dos dados para treinamento efetivo e 10% para um conjunto de validação, utilizado para o mecanismo de early stopping e para guiar a busca por melhores arquiteturas.
- O conjunto de teste original foi mantido intacto e utilizado exclusivamente para a avaliação final das arquiteturas encontradas.

2.2. Ferramentas e Ambiente Computacional

O desenvolvimento e a execução dos experimentos foram conduzidos num ambiente com sistema operacional **Windows 11**, utilizando a linguagem de programação **Python** na versão 3.12.5. O ambiente interativo de desenvolvimento foi o **Jupyter Notebook**.

A construção, o treinamento e a avaliação dos modelos de redes neurais foram realizados com o auxílio da biblioteca de aprendizado profundo **PyTorch**. Para as operações numéricas e a implementação dos algoritmos genéticos, foi utilizada a biblioteca **NumPy**. Adicionalmente, a biblioteca **Matplotlib** foi empregada para a geração dos gráficos de resultados. Com o objetivo de acelerar o processo de treinamento, os experimentos foram executados em uma Unidade de Processamento Gráfico (GPU - RTX 3060 de 12GB VRAM) por meio da plataforma **CUDA** versão 12.8 da NVIDIA.

2.3. Arquiteturas e Espaço de Busca

A eficácia de uma Rede Neural Convolucional (CNN) depende fundamentalmente de sua arquitetura. Uma CNN típica é composta por uma sequência de camadas de convolução, agrupamento (*pooling*) e totalmente conectadas (*fully-connected*). As camadas convolucionais aplicam um conjunto de filtros para extrair mapas de características (*feature maps*) da entrada, enquanto as camadas de *pooling* reduzem a dimensionalidade espacial dos mapas. O projeto manual dessas redes é um desafio, pois envolve uma vasta gama de decisões, como o número e a ordem das camadas, além dos hiperparâmetros de cada uma (tamanho do kernel, *stride*, etc.). Esse processo, além de lento e caro, exige profundo conhecimento técnico.

O objetivo do NAS é automatizar esse processo de descoberta. Para isso, é essencial definir um **espaço de busca** que delimite as arquiteturas que podem ser exploradas pelo algoritmo. No presente trabalho, em vez de projetar arquiteturas inteiras do zero, foi definida uma estrutura de CNN dinâmica, cujos componentes são parametrizados por um cromossomo genético.

2.3.1. Codificação da Arquitetura e Cromossomos

Para explorar o espaço de arquiteturas, cada rede candidata é codificada em um cromossomo de 10 bits. Esta representação binária de tamanho fixo serve como um genótipo que parametriza uma classe de CNNs dinâmicas [Xie and Yuille 2017]. A seguir, é detalhado como cada gene ou conjunto de genes é decodificado para definir os hiperparâmetros e a topologia da rede.

Gene 0: Otimizador Este gene binário seleciona o algoritmo de otimização para o treinamento da rede. O valor 0 corresponde ao otimizador **AdamW** com uma taxa de aprendizado de 0.001, enquanto o valor 1 seleciona o **SGD** (Gradiente Descendente Estocástico) com uma taxa de aprendizado de 0.01 e momento de 0.9. A escolha do otimizador impacta diretamente a dinâmica de convergência e a performance final do modelo.

Gene 1: Tamanho do Kernel Convolucional Determina o tamanho do filtro (kernel) em todas as camadas convolucionais. O valor 0 define um kernel de **3x3**, enquanto o valor 1 define um kernel de **5x5**. O preenchimento (*padding*) é ajustado automaticamente para preservar a dimensão espacial dos mapas de características.

Gene 2: Profundidade da Rede (Blocos) Controla a profundidade geral da parte convolucional da rede, definindo se ela será composta por **2 blocos** (valor 0) ou **3 blocos** (valor 1) sequenciais. Um bloco consiste em um conjunto de camadas convolucionais seguido por uma camada de *pooling*.

Gene 3: Tipo de Pooling Define a operação de *pooling* ao final de cada bloco convolucional. O valor 0 seleciona o **Max Pooling**, que extrai a característica mais proeminente de uma região, enquanto o valor 1 seleciona o **Average Pooling**, que calcula a média das características, tendo um efeito de suavização.

Gene 4: Estratégia das Camadas Densas Este gene define como a parte final da rede (o classificador) é construída. Com o valor 0 (*fix*), a rede adota uma estrutura onde a quantidade de neurônios é reduzida pela metade a cada camada densa subsequente. Com o valor 1 (*rand*), a dimensionalidade de cada camada densa é sorteada de uma lista pré-definida, introduzindo maior variabilidade estrutural.

Gene 5: Função de Ativação Convolucional Seleciona a função de ativação não-linear a ser aplicada após cada camada de convolução. O valor 0 corresponde à **ELU** (Exponential Linear Unit) e o valor 1 à **Leaky ReLU**. Ambas são variantes da ReLU.

Genes 6 e 7: Convoluções por Bloco Estes dois bits são decodificados conjuntamente para um número inteiro que define a complexidade interna de cada bloco convolucional. O valor resultante (de 1 a 4) especifica quantas camadas de convolução serão empilhadas dentro de cada um dos blocos definidos pelo Gene 2.

Genes 8 e 9: Profundidade do Classificador De forma similar aos anteriores, estes dois bits determinam o número de camadas densas (totalmente conectadas) ocultas que compõem o classificador da rede, variando de 1 a 4 camadas.

2.3.2. Estrutura da Rede Dinâmica

Com base no cromossomo decodificado, o modelo é construído seguindo uma macroarquitetura que consiste em duas partes principais:

1. **Rede Convolucional:** Uma sequência de blocos convolucionais. Cada bloco é composto pelo número de camadas de convolução definido pelos genes 6-7, seguido por uma única camada de *pooling* (Max ou Average). A função de ativação (ELU ou Leaky ReLU) é aplicada após cada convolução.
2. **Rede Totalmente Conectada:** Após a parte convolucional, uma camada *Flat-ten* transforma os mapas de características em um vetor. O tamanho deste vetor é determinado dinamicamente, garantindo flexibilidade. A este vetor, segue-se uma sequência de camadas densas ocultas, cujo número e dimensionalidade são definidos pelos genes 4, 8 e 9, finalizando com uma camada de saída de 10 neurônios (correspondente às 10 classes do CIFAR-10).

2.4. Estratégia de Busca e Otimização

Para navegar no espaço de busca de arquiteturas, é utilizada uma abordagem baseada em Algoritmos Genéticos. Foram comparadas duas estratégias: um AG tradicional com parâmetros fixos e a proposta, o LAC-NAS, com parâmetros adaptativos controlados pelo caos.

2.4.1. Algoritmo Genético Tradicional (Baseline)

A abordagem de referência segue a estrutura canônica de um Algoritmo Genético. O processo inicia com uma população de 20 indivíduos, onde cada indivíduo representa uma arquitetura de CNN codificada em um cromossomo de 10 bits. A qualidade de cada indivíduo é medida por sua função de *fitness*, que corresponde à acurácia do modelo no conjunto de teste após o treinamento. O algoritmo evolui por 10 gerações, aplicando os seguintes operadores:

- **Seleção:** Utiliza-se o método da roleta, onde a probabilidade de um indivíduo ser selecionado como pai para a próxima geração é proporcional ao seu valor de *fitness* (acurácia) normalizado.
- **Crossover:** É aplicado um operador de crossover de ponto único. Para cada par de pais, um ponto de corte é sorteado no cromossomo, e as caudas dos cromossomos são trocadas para gerar dois filhos [Alam et al. 2020]. Esta operação ocorre baseada numa taxa de crossover fixa.
- **Mutação:** Após o crossover, cada bit no cromossomo de cada novo indivíduo tem uma chance de ser invertido (*bit-flip*), de 0 para 1 ou vice-versa [Alam et al. 2020]. Esta operação ocorre baseada numa taxa de mutação fixa.

Nesta abordagem tradicional, a principal limitação é que as taxas de crossover e mutação são constantes durante toda a execução, o que pode levar à pontos de ótimos locais e, por consequência, convergência prematura.

2.4.2. A Abordagem Proposta: LAC-NAS

Para superar a rigidez do AG tradicional, o LAC-NAS introduz um mecanismo adaptativo para as taxas de crossover e mutação, utilizando conceitos da teoria do caos.

Em vez de taxas fixas, as probabilidades de crossover e mutação são geradas dinamicamente a cada geração pelo mapa logístico, uma equação clássica que pode exibir comportamento caótico. A equação é definida como:

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n) \quad (2)$$

onde $x_n \in [0, 1]$ é o valor da taxa (crossover e mutação) na geração n , e r é um parâmetro que controla o comportamento do sistema. Ao iterar esta equação, geramos valores pseudoaleatórios e caóticos para a probabilidade de crossover e mutação, permitindo que a intensidade da exploração e da exploração varie de forma não-linear.

O LAC-NAS também conta com o controle dinâmico do parâmetro r do mapa logístico. Para refinar o balanço entre exploração e exploração ao longo do tempo, o algoritmo adota uma estratégia de duas fases, ajustando o nível de caos desejado:

- **Fase de Exploração (Gerações 0-4):** O sistema busca um nível de caos moderado, com o expoente de Lyapunov (λ) alvo na faixa de **[0.3, 0.5]**. Isso incentiva uma busca mais ampla e diversificada no espaço de soluções, aumentando a probabilidade de descobrir regiões promissoras.
- **Fase de Exploração (Gerações 5 em diante):** O alvo é ajustado para uma faixa de caos baixo, com λ entre **[0.0, 0.2]**. Essa transição para um regime próximo da estabilidade foca o algoritmo no refinamento de soluções promissoras encontradas na fase inicial, realizando uma busca local mais precisa.

Ao final de cada geração, se o λ calculado estiver fora da faixa alvo da fase atual, o parâmetro r é ajustado em incrementos de ± 0.05 para guiar o sistema de volta ao nível de caos desejado. O valor de r é sempre mantido dentro dos limites de $[3.5, 3.7]$ para garantir um comportamento dinâmico controlado, na fronteira do caos. Este ciclo de feedback em duas fases permite que o LAC-NAS primeiro explore o espaço de busca de forma abrangente e depois refine as melhores soluções encontradas de forma convergente.

2.4.3. Cálculo do Expoente de Lyapunov

Para controlar dinamicamente o comportamento do mapa logístico, é essencial quantificar seu nível de caos. Isso é feito através do cálculo do **expoente de Lyapunov** (λ). Um expoente positivo ($\lambda > 0$) é um forte indicador de comportamento caótico.

A função se baseia na definição matemática do expoente de Lyapunov para um mapa unidimensional, como o mapa logístico, que é definido pela equação:

$$f(x) = r \cdot x \cdot (1 - x) \quad (3)$$

O expoente de Lyapunov (λ) é o valor médio do logaritmo da derivada da função ao longo da trajetória do sistema [Jüngling and Kinzel 2012]. Formalmente, é definido pelo limite:

$$\lambda = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=0}^{N-1} \ln |f'(x_i)| \quad (4)$$

onde x_i é o valor da série na iteração i , e $f'(x_i)$ é a derivada da função do mapa logístico no ponto x_i . A derivada é:

$$f'(x) = \frac{d}{dx}(rx - rx^2) = r - 2rx = r(1 - 2x) \quad (5)$$

3. Experimentos e Resultados

Para avaliar e comparar o desempenho da abordagem LAC-NAS com um algoritmo genético tradicional, foi desenhado um conjunto de experimentos cujas configurações são detalhadas a seguir.

3.1. Configuração Experimental

Ambas as abordagens, o Algoritmo Genético tradicional (baseline) e o LAC-NAS, foram executadas sob as mesmas condições gerais para garantir uma comparação justa.

Os parâmetros fundamentais para a execução do algoritmo evolucionário foram definidos da seguinte forma:

- **Tamanho da População:** 20 indivíduos.
- **Número de Gerações:** 10 gerações.
- **Tamanho do Cromossomo:** 10 bits, conforme detalhado na Seção 2.3.1.
- **Seleção:** Método da roleta, onde a aptidão de cada indivíduo determina sua probabilidade de seleção.
- **Operadores Genéticos:** Crossover de ponto único e mutação do tipo *bit-flip*.

O *fitness* de cada indivíduo foi definido como a acurácia final obtida no conjunto de teste do CIFAR-10. O processo para calcular este valor foi o seguinte:

- **Treinamento por Épocas:** Cada arquitetura foi treinada por um máximo de 20 épocas.
- **Early Stopping:** Para acelerar o processo e evitar o sobreajuste, foi utilizado um mecanismo de parada antecipada (*early stopping*). O treinamento de um indivíduo era interrompido se a perda de validação não apresentasse uma melhoria de pelo menos 0.01 por 3 épocas consecutivas.
- **Função de Perda:** A função de perda utilizada em todos os treinamentos foi a `CrossEntropyLoss`.

3.1.1. Abordagem de Referência

Para a linha de base comparativa, o algoritmo tradicional foi configurado com taxas de crossover e mutação fixas durante toda a execução. Os valores foram definidos com base em convenções comuns na literatura:

- Taxa de Crossover Fixa: 60%
- Taxa de Mutação Fixa: 5%

3.1.2. Abordagem Proposta

A configuração do LAC-NAS difere apenas na maneira como as taxas de crossover e mutação são tratadas, utilizando o controle caótico descrito na metodologia:

- **Parâmetro inicial de caos (r):** O valor inicial de r no mapa logístico foi definido como um valor aleatório entre 3.50 e 3.70.
- **Controle do Expoente de Lyapunov (λ):** A faixa alvo de λ foi definida em duas fases:
 - Gerações 0-4 (Exploração): $\lambda \in [0.3, 0.5]$.
 - Gerações 5 em diante (Exploração): $\lambda \in [0.0, 0.2]$.
- **Ajuste e Limites de r :** O parâmetro r é ajustado em ± 0.05 a cada geração para manter λ na faixa desejada, sendo sempre limitado ao intervalo $[3.5, 3.7]$.

3.2. Resultados: Algoritmo Genético Tradicional

A execução do algoritmo genético tradicional, com taxas de crossover e mutação fixas, foi realizada por 10 gerações. Os resultados da melhor acurácia encontrada em cada geração foram compilados e são apresentados na Tabela 1 e visualizados no gráfico de convergência da Figura 1.

Table 1. Melhor acurácia de teste por geração para o AG Tradicional.

Geração	Melhor Acurácia (%)
0	76.53
1	77.05
2	78.24
3	77.94
4	78.19
5	76.77
6	76.00
7	76.04
8	75.88
9	76.75

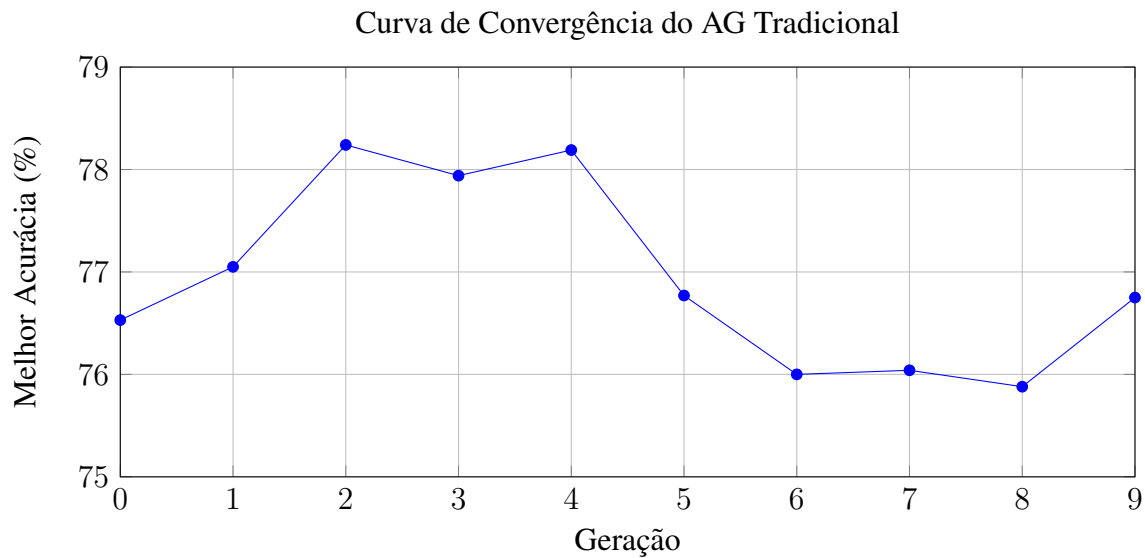


Figure 1. Evolução da melhor acurácia encontrada ao longo das gerações pelo AG tradicional.

3.2.1. Análise dos Resultados do AG Tradicional

A análise dos resultados revela um comportamento característico de algoritmos evolucionários com parâmetros fixos. Observa-se um progresso inicial rápido, com a melhor acurácia global de **78.24%** sendo alcançada logo na segunda geração. A arquitetura correspondente a este pico de desempenho foi o Modelo 12 da Geração 2.

No entanto, após atingir esse pico, o algoritmo não conseguiu encontrar soluções superiores nas 7 gerações subsequentes. A melhor acurácia por geração oscilou, mas nunca ultrapassou o máximo encontrado na Geração 2. Este comportamento é um forte indicativo de convergência prematura. A população do algoritmo provavelmente perdeu diversidade e convergiu para um ótimo local, do qual não conseguiu escapar. As taxas fixas de mutação e crossover, embora eficazes no início, mostraram-se insuficientes para introduzir a variabilidade necessária para explorar outras regiões do espaço de busca e superar o máximo local encontrado.

3.3. Resultados: LAC-NAS

A abordagem proposta foi executada sob as mesmas condições experimentais do algoritmo tradicional, com a diferença de que as taxas de crossover e mutação foram controladas dinamicamente pelo sistema caótico. Os resultados da melhor acurácia obtida em cada geração são apresentados na Tabela 2 e na Figura 2.

3.3.1. Análise dos Resultados do LAC-NAS

A trajetória de convergência do LAC-NAS apresenta um comportamento distinto. Assim como o AG tradicional, o algoritmo exibe um ganho de desempenho inicial, atingindo sua acurácia máxima de **77.94%** na Geração 2. Este resultado foi obtido pelo Modelo 20 da Geração 2.

Table 2. Melhor acurácia de teste por geração para o LAC-NAS.

Geração	Melhor Acurácia (%)
0	76.79
1	77.14
2	77.94
3	74.73
4	76.80
5	75.50
6	75.29
7	76.69
8	76.87
9	75.09

Curva de Convergência do LAC-NAS

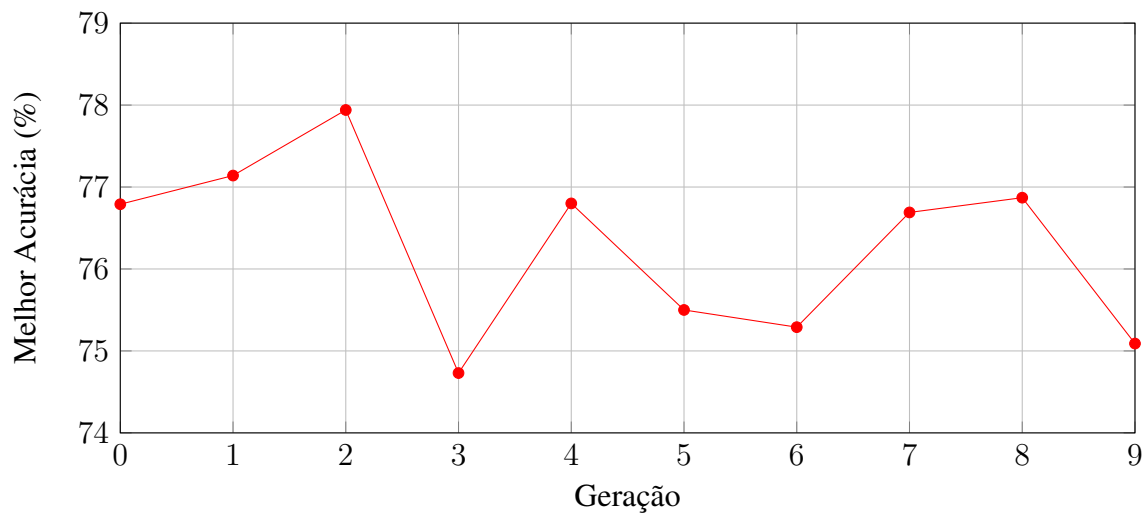


Figure 2. Evolução da melhor acurácia encontrada ao longo das gerações pelo LAC-NAS.

Após o pico inicial, o algoritmo entra em um platô de desempenho, não conseguindo superar a marca estabelecida. É interessante notar que o pico de acurácia foi alcançado durante a **fase de exploração** (gerações 0-4), quando o sistema operava com um nível de caos mais elevado. Ao transicionar para a **fase de exploração** (gerações 5 em diante), com um nível de caos mais baixo, o algoritmo manteve uma performance relativamente estável, porém sem novos picos.

Isso sugere que, embora a estratégia de duas fases tenha funcionado como projetado, a fase de exploração pode não ter descoberto uma região do espaço de busca com um potencial de ótimo global superior ao do AG tradicional nesta execução específica.

4. Conclusões

Este artigo propôs e avaliou o LAC-NAS, uma nova abordagem para a Busca Automatizada de Arquiteturas Neurais (NAS) que emprega um Algoritmo Genético com parâmetros de busca controlados dinamicamente. O objetivo era superar a limitação de

AGs tradicionais com taxas de crossover e mutação fixas, que frequentemente levam à convergência prematura. A hipótese central era que, ao utilizar o mapa logístico para gerar taxas dinâmicas e o expoente de Lyapunov para controlar o nível de caos do sistema em duas fases (exploração e exploração), o LAC-NAS alcançaria um melhor equilíbrio entre a busca global e o refinamento local, resultando em arquiteturas de maior desempenho.

Os experimentos foram conduzidos comparando o LAC-NAS com um AG tradicional sob as mesmas condições, utilizando o dataset CIFAR-10. A análise dos resultados revelou que, nesta execução experimental específica, a hipótese inicial não foi confirmada em termos de desempenho máximo. O AG tradicional atingiu uma acurácia máxima de **78.24%**, enquanto o LAC-NAS alcançou um pico de **77.94%**. Ambos os algoritmos demonstraram um comportamento semelhante, encontrando suas melhores soluções no início do processo e posteriormente estagnando num ótimo local.

A interpretação desses resultados sugere que, embora o mecanismo de controle caótico do LAC-NAS seja teoricamente mais sofisticado, o resultado de um processo evolucionário é estocástico. É possível que a execução do AG tradicional tenha sido beneficiada por uma combinação inicial de indivíduos e operadores genéticos que a levou a um ótimo local ligeiramente superior. Por outro lado, o LAC-NAS, apesar de sua dinâmica de busca mais elaborada com fases de exploração e exploração, pode não ter explorado a região do espaço de busca que continha a melhor solução encontrada pelo seu concorrente. Isso evidencia que a eficácia de uma estratégia de busca adaptativa também é sensível aos seus próprios meta-parâmetros, como as faixas de caos definidas para o expoente de Lyapunov e o momento da transição entre as fases.

É fundamental reconhecer as limitações deste estudo. A comparação foi baseada em uma única execução de cada algoritmo, o que não permite uma validação estatística robusta. Além disso, o número de gerações (10) pode ter sido insuficiente para que as vantagens da busca adaptativa do LAC-NAS se manifestassem plenamente, especialmente na fase de exploração.

Como trabalhos futuros, sugerem-se as seguintes direções:

1. **Validação Estatística:** Realizar múltiplas execuções independentes de ambos os algoritmos para comparar seus desempenhos médios e desvios padrão, oferecendo conclusões mais robustas.
2. **Execuções Prolongadas:** Aumentar o número de gerações para observar se o LAC-NAS consegue, em um prazo maior, escapar de ótimos locais onde o AG tradicional permanece estagnado.

Apesar de não ter superado o baseline em acurácia máxima nesta instância, o LAC-NAS representa uma contribuição ao introduzir um novo mecanismo de controle de busca. Esta primeira abordagem pode não ter se mostrado promissora, mas sua investigação aprofundada pode revelar configurações que seu potencial de adaptação se traduza numa clara e consistente vantagem.

References

- Alam, T., Qamar, S., Dixit, A., and Benaida, M. (2020). Genetic algorithm: Reviews, implementations, and applications.
- Jüngling, T. and Kinzel, W. (2012). Scaling of lyapunov exponents in chaotic delay systems.
- Liashchynskyi, P. and Liashchynskyi, P. (2019). Grid search, random search, genetic algorithm: A big comparison for nas.
- Liu, H., Simonyan, K., and Yang, Y. (2019). Darts: Differentiable architecture search.
- Sun, Y., Xue, B., Zhang, M., Yen, G. G., and Lv, J. (2020). Automatically designing cnn architectures using the genetic algorithm for image classification. *IEEE Transactions on Cybernetics*, 50(9):3840–3854.
- White, C., Safari, M., Sukthanker, R., Ru, B., Elsken, T., Zela, A., Dey, D., and Hutter, F. (2023). Neural architecture search: Insights from 1000 papers.
- Xie, L. and Yuille, A. (2017). Genetic cnn.
- Xue, C., Li, J., Wang, X., Zhan, Y., Yan, J., and Li, C.-G. (2025). On neural architecture search and hyperparameter optimization: A max-flow based approach. *Neural Networks*, 188:107507.